بازی سازی

موتور بازی‌سازی یونیتی چیست؟

- یونیتی، نامی آشنا در جهان توسعه بازی‌ها، به عنوان یک موتور بازی‌سازی قدرتمند و محبوب شناخته می‌شود که توسعه‌دهندگان را قادر می‌سازد تا بازی‌های دو بعدی و سه بعدی خود را برای طیف گسترده‌ای از پلتفرم‌ها از جمله وب، موبایل، کنسول‌های بازی و حتی VR توسعه دهند.
- این پلتفرم نه تنها به دلیل انعطاف‌پذیری بالا، بلکه به خاطر محیط توسعه یکپارچه و کاربرپسند خود مورد توجه قرار گرفته است.

- یونیتی یک موتور بازی‌سازی است که امکان ساخت بازی‌ها و تجربیات تعاملی را در ابعاد و پلتفرم‌های مختلف فراهم می‌کند.

- این ابزار نه تنها قابلیت‌های قدرتمندی برای رندرینگ گرافیک، شبیه‌سازی فیزیک و پشتیبانی از شناسایی برخوردها (Collision) دارد، بلکه محیطی چندمنظوره برای توسعه‌دهندگان بازی فراهم می‌آورد.

ویژگی‌های Unity

۱- قابلیت کراس - پلتفرم :

یکی از برجسته‌ترین ویژگی‌های یونیتی، قابلیت کراس - پلتفرم آن است. این ویژگی به توسعه‌دهندگان اجازه می‌دهد تا با یک پروژه‌ی واحد، بازی یا برنامه‌های خود را برای انواع مختلفی از دستگاه‌ها مانند کامپیوترهای شخصی، کنسول‌های بازی، تلفن‌های هوشمند و حتی پلتفرم‌های واقعیت مجازی طراحی و توسعه دهند.

۲- پشتیبانی از فیزیک واقع‌گرا :

یونیتی با ارائه شبیه‌سازی‌های فیزیکی پیچیده و واقع‌گرایانه، انقلابی در صنعت بازی‌های ویدئویی ایجاد کرده است. این موتور بازی‌سازی از موتور فیزیکی قدرتمند PhysXنیویدیا استفاده می‌کند که به طور دقیق برخوردها، جاذبه و سایر پدیده‌های فیزیکی را شبیه‌سازی می‌کند و تجربه‌ای بی‌نظیر و واقعی را برای گیمرها رقم می‌زند.

ویژگی‌های Unity

۳- ادغام آسان با سیستم‌های مختلف :

یونیتی امکان ادغام آسان با طیف وسیعی از سیستم‌ها و ابزارهای توسعه را فراهم می‌کند، از جمله سیستم‌های کنترل نسخه، ابزارهای ساخت و تست خودکار، و پلتفرم‌های شخص ثالث. این ویژگی توسعه‌دهندگان را قادر می‌سازد تا به راحتی اجزای مختلف پروژه‌های خود را با یکدیگر همگام سازند.

۴- ابزارهای انیمیشن پیشرفته :

یونیتی ابزارهای انیمیشن سازی پیشرفته‌ای را ارائه می‌دهد که به توسعه‌دهندگان اجازه می‌دهد تا انیمیشن‌های دقیق و جذابی را برای شخصیت‌ها و سایر المان‌های بازی خلق کنند. این ابزارها شامل تایم‌لاین‌ها، سیستم‌های کنترل دقیق حرکت و امکان وارد کردن انیمیشن‌های ساخته شده در نرم‌افزارهای دیگر می‌باشد.

ویژگی‌های Unity

۵- پشتیبانی از گرافیک دو بعدی و سه بعدی:

یونیتی از گرافیک‌های دو بعدی و سه بعدی پشتیبانی می‌کند و به توسعه‌دهندگان امکان می‌دهد که سبک هنری دلخواه خود را برای پروژه‌ها انتخاب کنند. این پشتیبانی شامل ابزارهای قدرتمند برای ساخت سایه‌بان‌ها، سفارشی‌سازی نورپردازی، و افزودن جلوه‌های بعد از پردازش است. یونیتی این امکان را می‌دهد که زمین‌های سه‌بعدی ایجاد کنید و نقشه‌های دوبعدی را به طور دقیق در موتور بازی طراحی کنید.

۶- قابلیت اجرا روی هر نوع پلتفرم:

یونیتی پشتیبانی گسترده‌ای از پلتفرم‌ها دارد، از جمله Android، iOS، Windows، MacOS، Linux، و کنسول‌های بازی مانند PSو Xbox One. توسعه‌دهندگان می‌توانند بازی‌ها را با تغییرات اندک بین پلتفرم‌ها منتقل کنند، که این امر توسعه‌ی کراس‌پلتفرم را تسهیل می‌بخشد.

ویژگی‌های Unity

۷- واقعیت مجازی و قابلیت افزوده:

یونیتی از پیشروان حمایت از واقعیت مجازی و واقعیت افزوده بوده و پکیج‌های متعددی برای استفاده از این تکنولوژی‌ها ارائه می‌دهد. این قابلیت‌ها به‌خصوص برای بازی‌های موبایل که از واقعیت افزوده استفاده می‌کنند، مفید است.

۸- فروشگاه Assetهای گسترده:

یونیتی یک فروشگاه گسترده از Assetها دارد که توسعه‌دهندگان می‌توانند از آن برای سرعت بخشیدن به روند توسعه بازی‌ها استفاده کنند. این فروشگاه شامل کاراکترها، تصاویر، صداها و موسیقی‌های آماده‌ای است که می‌توانند بلافاصله در پروژه‌ها استفاده شوند.

مقایسه یونیتی با سایر موتورهای بازی سازی

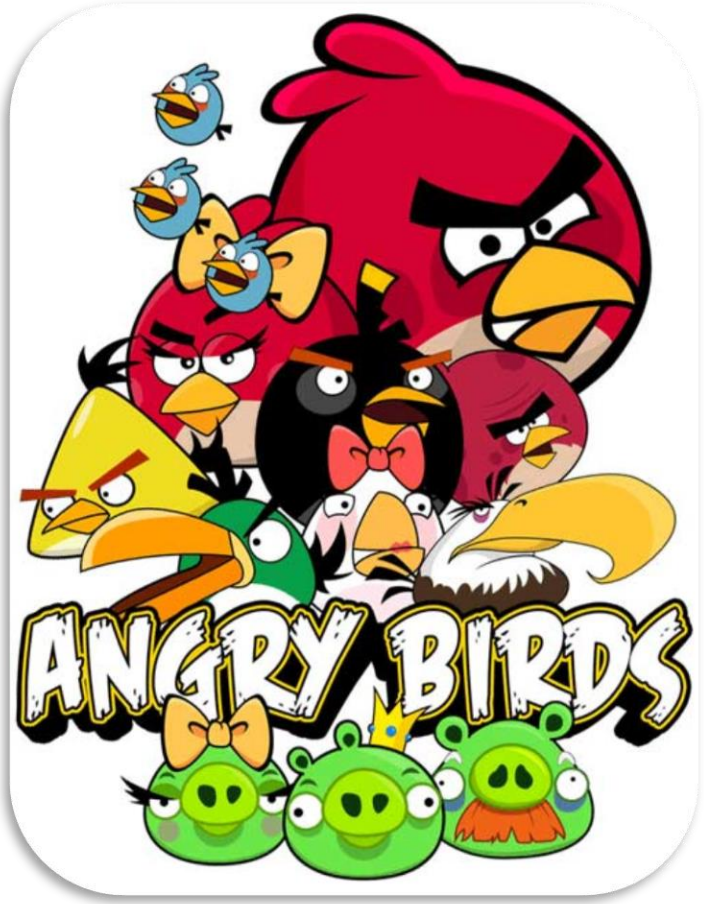
موتور بازی	منحنی یادگیری	زبان	2D/3D
Unity	۴ از ۵	C#, JS, Boo	هر دو
Unreal	۵ از ۵	C++	3D
CryEngine	۵ از ۵	C++, Lua	هر دو
Game Salad	۱ از ۵	GML	2D
Game Maker	۱ از ۵	N/A	2D
Construct 2	۲ از ۵	JS	2D

بازی های ساخته شده با یونیتی









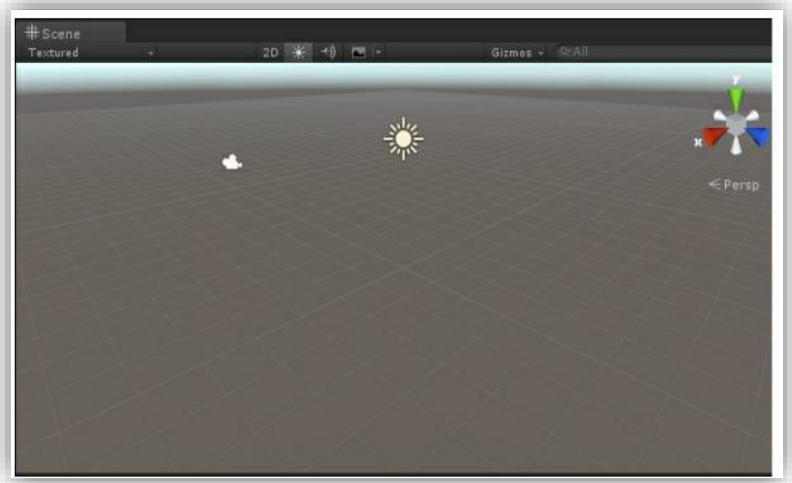
محیط یونیتی

انواع نما (view)

- یونیتی امکان دیدن اجزای خاص یک پروژه را فراهم کرده است.این کار از طریق نماها (view) انجام میشود.
- ۵ نما در یونیتی وجود دارد :

۱. Scene
۲. Game
۳. Hierarchy
۴. Project
۵. Inspector

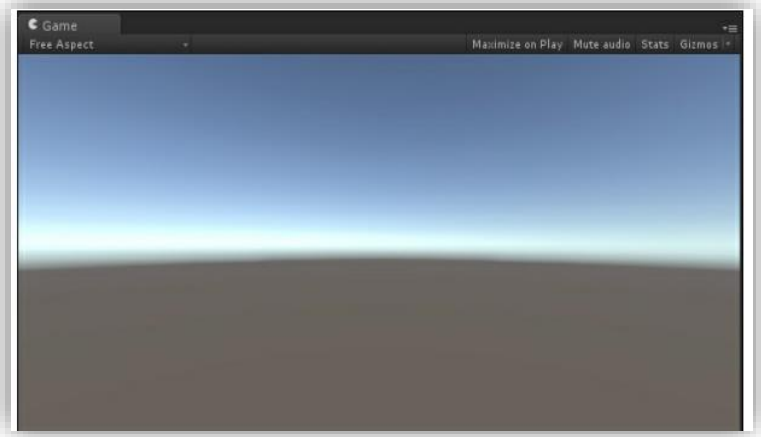
scene

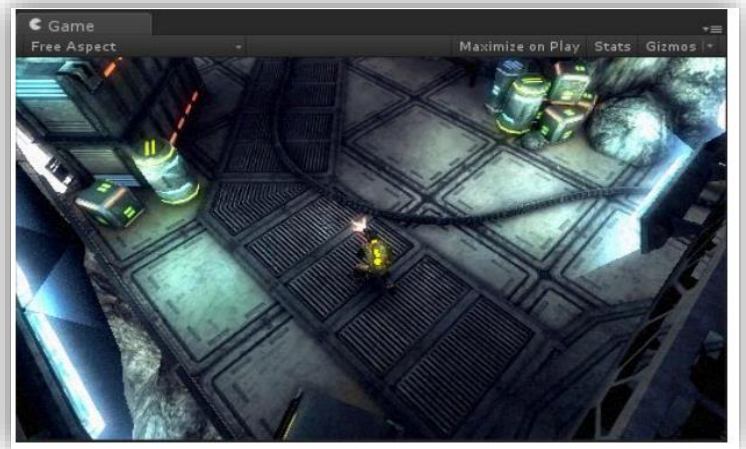




- صحنه ها در یونیتی ، حکم مراحل بازی هستند.
- برای هر مرحله از بازی،صحنه های متفاوتی خواهید داشت.
- در این نما میتوانید محیط، دشمنان،دوربین ها،بازیکن و تمام اشیاءموردنظر خود را بچینید.

Game

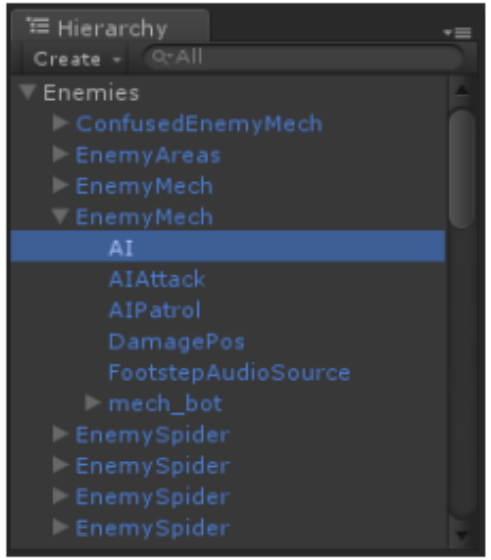


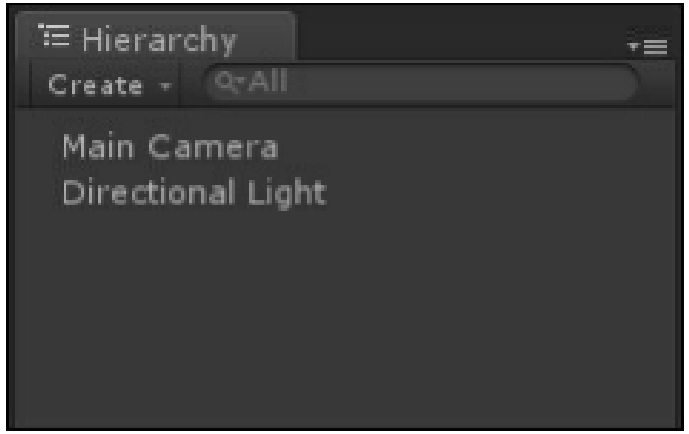


- این نما،جایی است که بازی در هنگام ساخته شدن،اجرا میشود.
- نمایی از خروجی بازی که توسط یک دوربین گرفته میشود است.این نما شامل دکمه های شروع ،توقف ومکث بازی است.
- در یونیتی میتوان در داخل موتور بازی،بازی را اجرا کرد.(یعنی لازم نیست ابتدا بازی را کامپایل کرده و در خارج از یونیتی ان را اجرا کرد.)
- بازی ها معمولا از دید سوم شخص اجرا میشوند.به این معنا که دوربین در بالا یا پشت سر کاراکتر بازی قرار میگیرد.

Hierarchy

- در اینجا یک لیست از اشیاءموجود درصحنه را میتوان مشاهده کرد.
- این نما قابلیت جستجوی یک شئ در Scene را نیز دارا میباشد.
- اگر یک شئ را از Scene پاک کنید،متقابلا از Hierarchy هم پاک میشود.





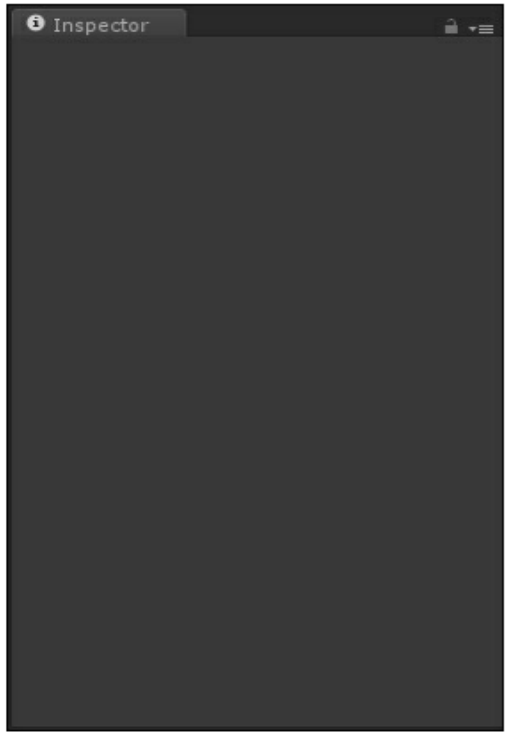
Project

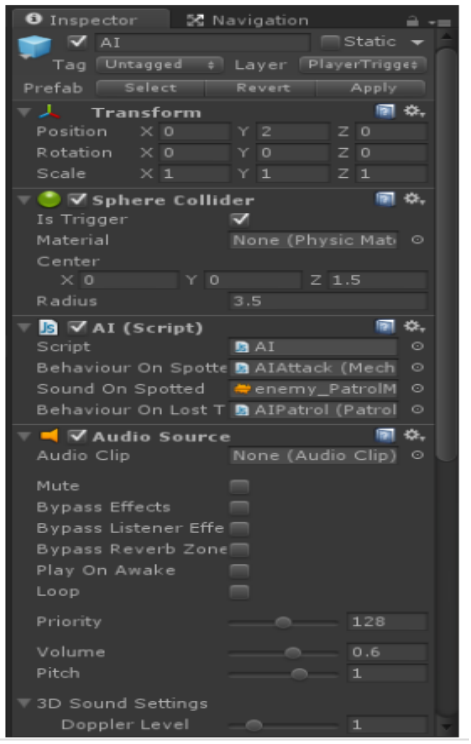
- تمام چیزهایی که در کل پروژه وجود دارد، از جمله اشیا،گرافیک ها و اسکریپت ها و... را نمایش میدهد.
- این بخش دارای دوستون Favorites و Assets میباشد.





Inspector

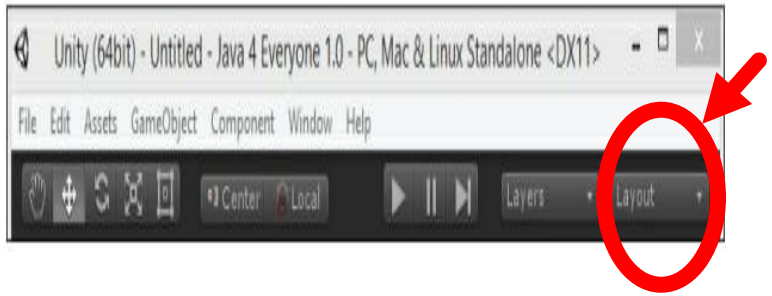




- این بخش خصوصیات(جزییات) یک شی را نمایش میدهد.
- همچنین در اینجا میتوانید تغییراتی را روی یک شی اعمال کنید.
- به اطلاعات ثابت این تب،میتوان به اطلاعات مربوط به قرارگیری شی،چرخش واندازه آن اشاره کرد.
- هرکامپوننتی که به شیءاضافه شود،به تب نیز اضافه میشود.

طرح بندی(چیدمان) نماها(Layout)

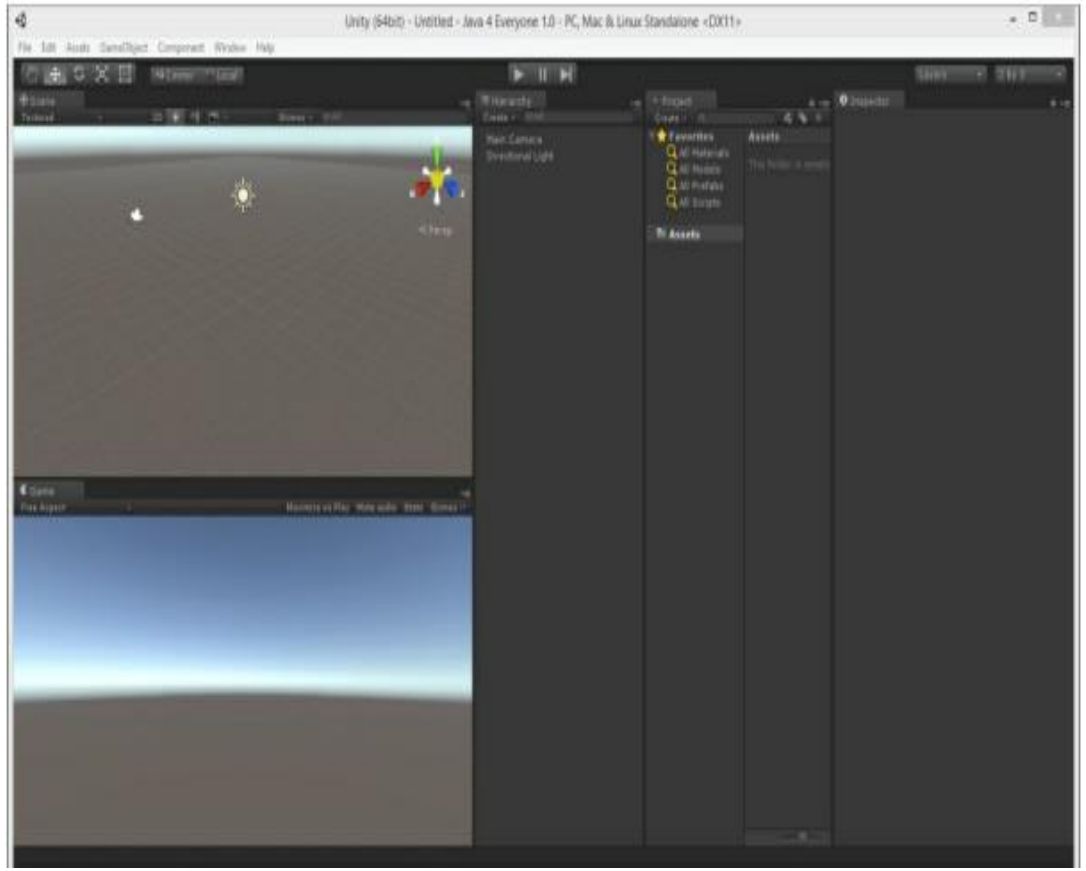
- یکی از امکاناتی که یونیتی دارد،این است که شما میتوانید شیوه چینش وطرح بندی محیط کاری موتور را تغییر دهید، به نحوی که میتوانید از پیش فرض ها استفاده کنید ویا اینکه خودتان آن را ایجاد کنید.
- طرح بندی وچینش ها اشاره به نحوه ی قرارگرفتن نماهای مختلف یونیتی روی صفحه دارد.
- برای استفاده از چینش های پیش فرض، گزینه **layout** ، در قسمت بالا و سمت راست قراردارد.
- اغلب کاربران فقط در یک چینش کار نمیکنند.
- چینش های مختلف،مزایای متفاوتی دارند و برای کارهای مختلف مناسب هستند.
- با درگ کردن میتوان چینش ها را تغییر داد.
- پس از انجام تغییرات در چینش ها،گزینه **Layout** و سپس **Save layout** را انتخاب کنید.



انواع طرح بندی

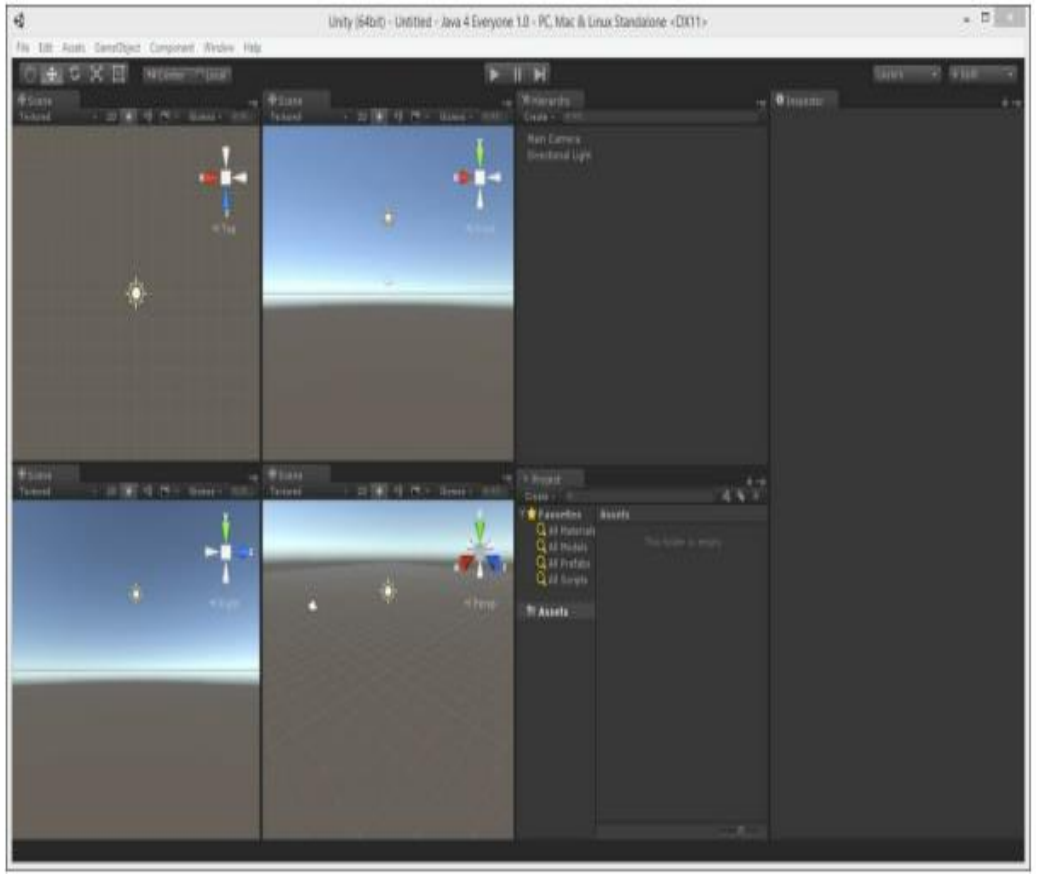
۱. 2 by 3
۲. 4 split
۳. Tall
۴. Wide
۵. Default

2 by 3



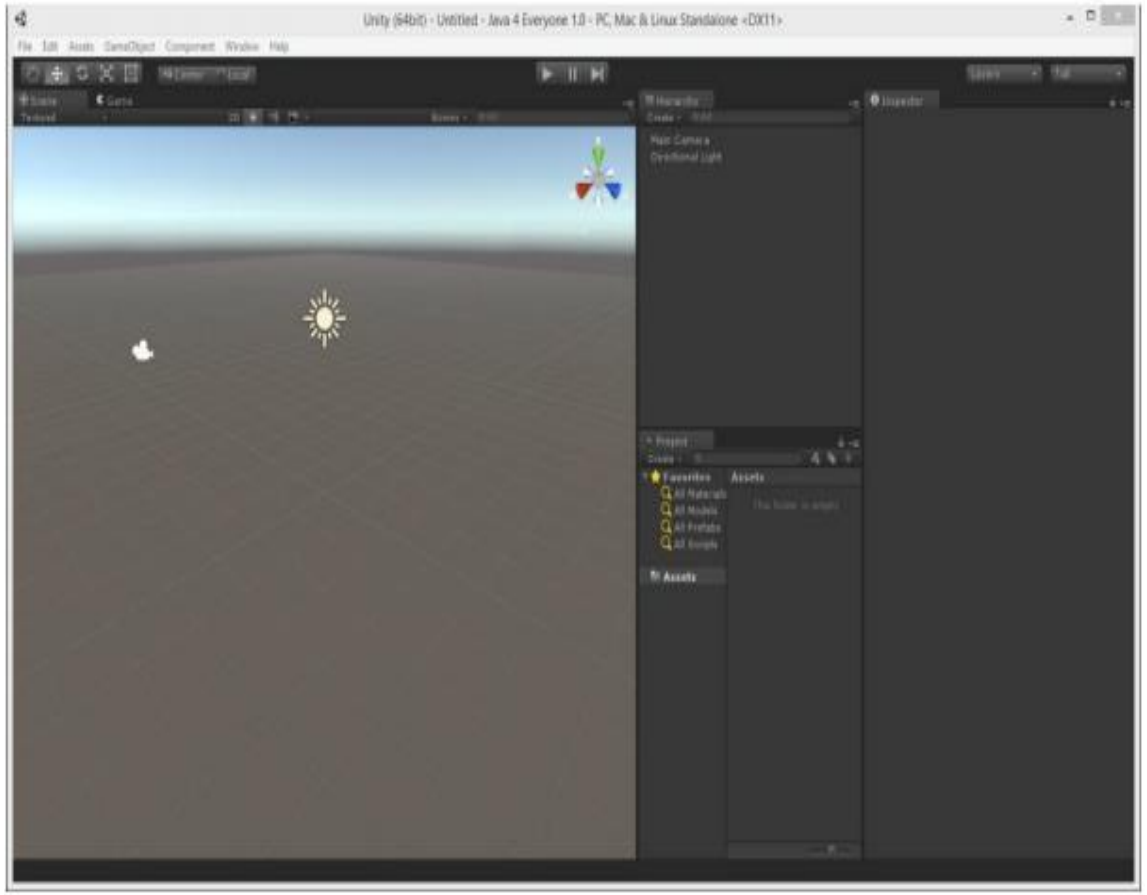
- در این چینش ، نماهای Scene و Geme در سمت چپ و نماهای Project و Hierarchy در وسط و Inspector در سمت راست قرار میگیرند.

4 Split



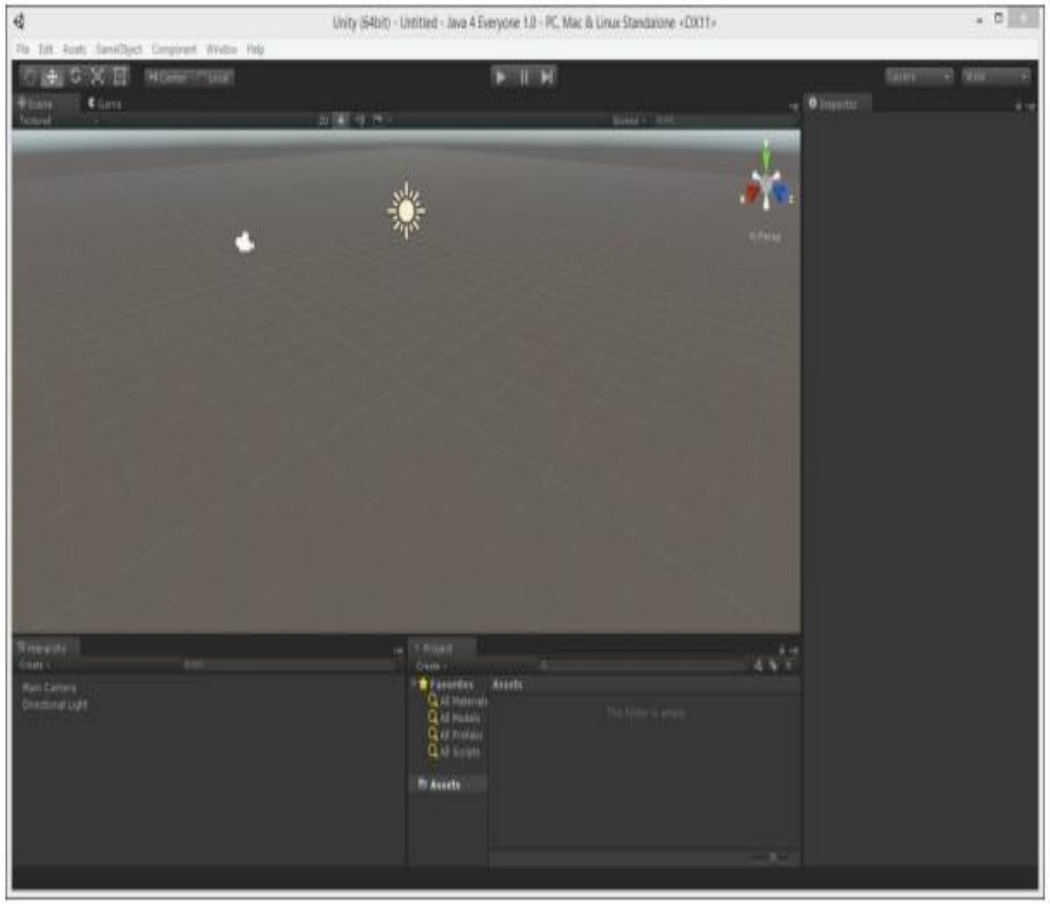
- ۴ نمای مختلف از یک صحنه را نمایش میدهد.
- با استفاده از این چینش، میتوان یک مرور عالی بر روی نحوه پیاده سازی نورپردازی و سایه زنی بازی انجام داد.

Tall



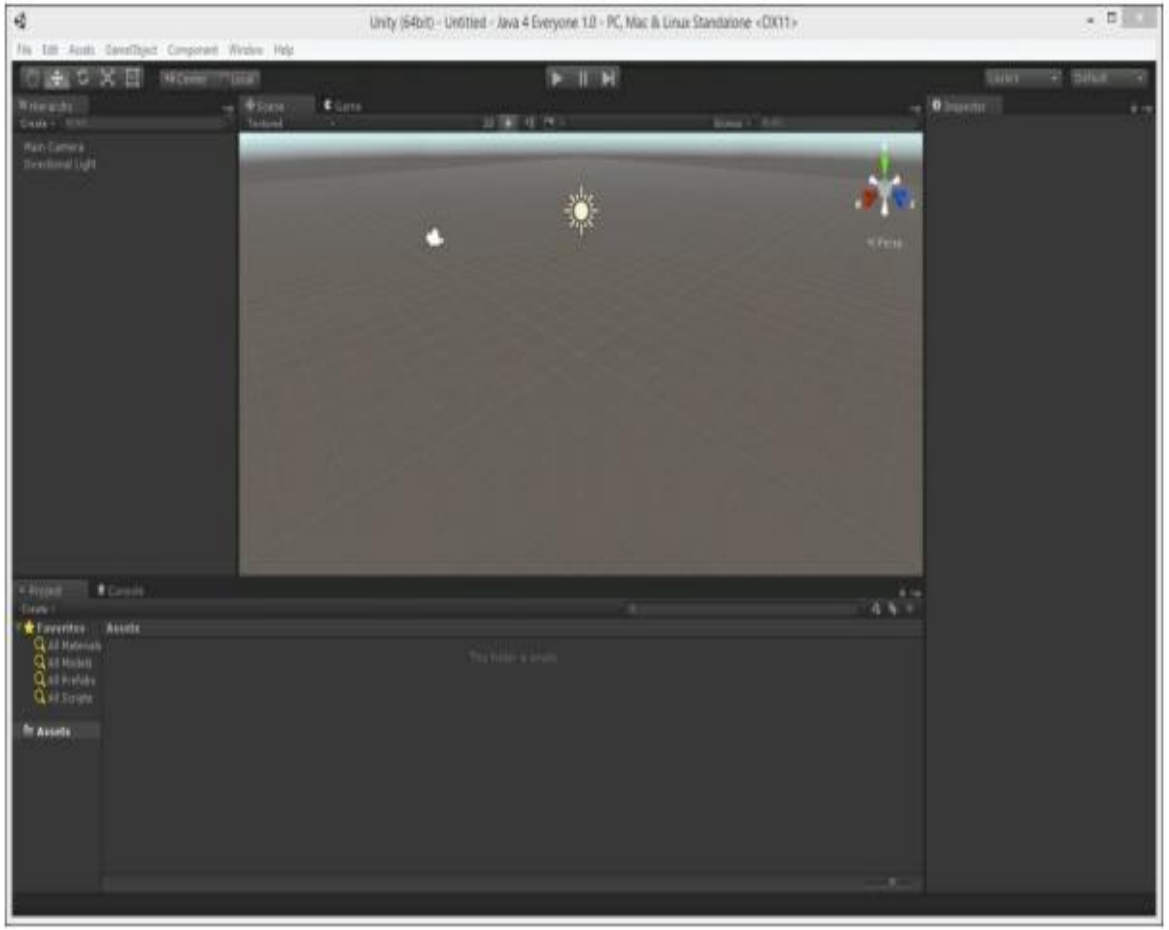
- یک تصویر بلند اما نه چندان وسیع از نمای Scene فراهم میکند.

Wide



- این چینش یک دید وسیع و پهن از نمای Scene را فراهم میکند.

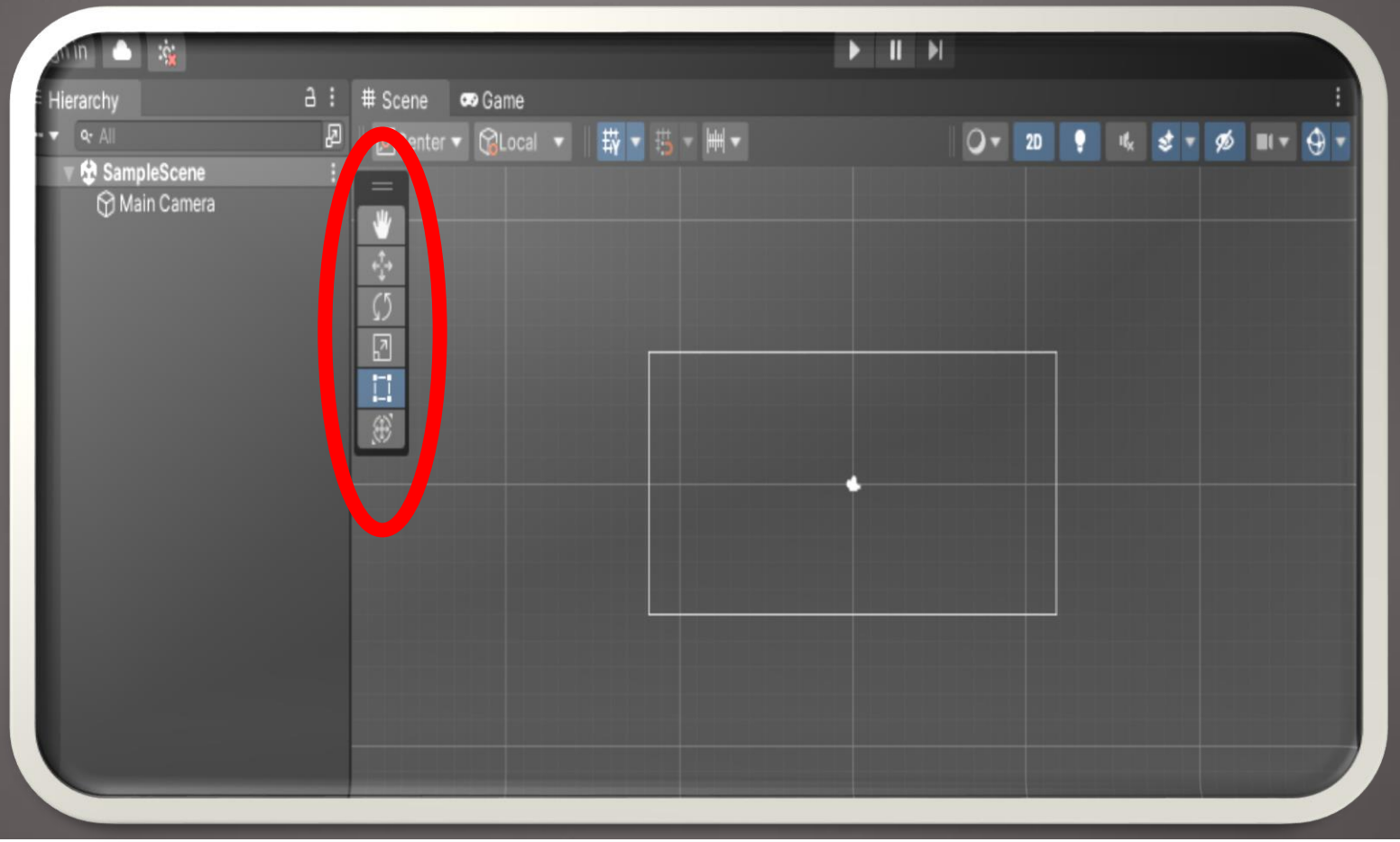
Default



- چینش **Default** ،نوعی از چینش **Wide** است.

- تنها تفاوت این است که در چینش **Default**، نمای **Hierarchy** در سمت **چپ** است.

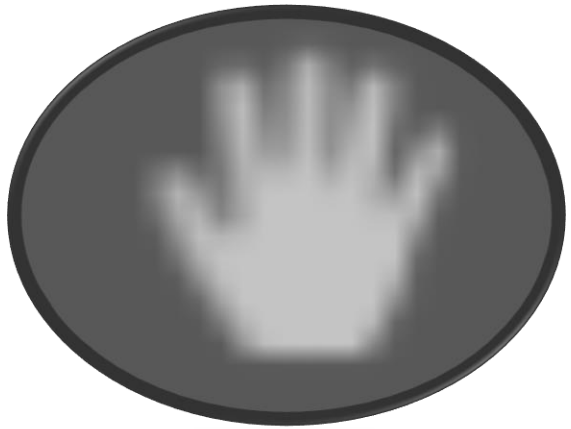
ابزارهای تغییرشکل



- این ابزارها در یونیتی به ما اجازه میدهند با نمای **Scene** تعامل داشته باشیم،زمین را ویرایش کنیم،اشیا را جابه جا کنیم و تغییرات را انجام دهیم.
- این ابزارها در گوشه بالا سمت چپ محیط کاری یونیتی واقع شده اند.
- این ابزارها عبارتند از:
  ۱. **View Tool**
  ۲. **Move Tool**
  ۳. **Rotate Tool**
  ۴. **Scale Tool**
  ۵. **Rect Tool**
  ۶. **Transform Tool**

View Tool

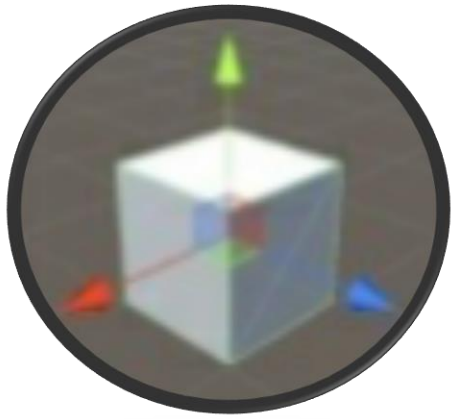
- با کمک این ابزار ،میتوانیم با **Scroll** (چرخاندن) موس ، به داخل صحنه زوم کنیم.
- با کلیک روی موس هم میتوان به اطراف صحنه حرکت کرد.



Move Tool

- این مورد برای جابجایی اشیاء در دنیای بازی در اطراف استفاده می شود.

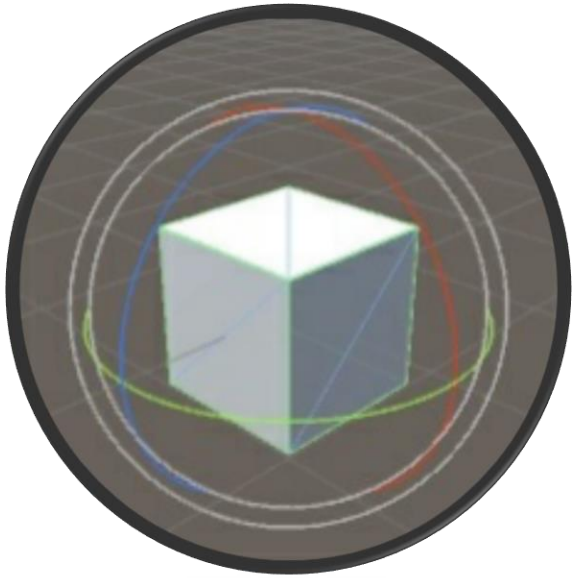




Rotate Tool

- از ابزار Rotate برای چرخاندن اشیاء در امتداد محور Z در دنیای بازی (یا شی اصلی) استفاده میکنیم.

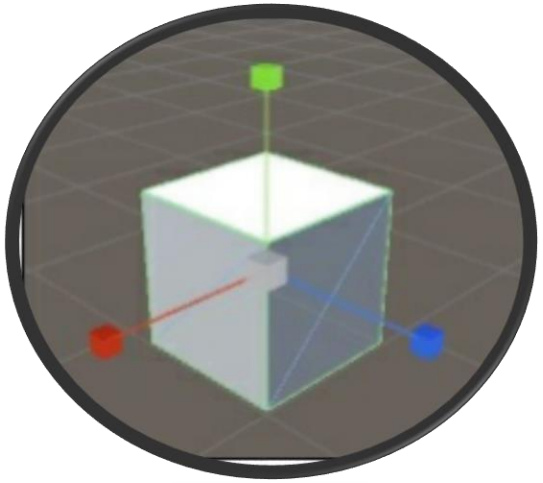




Scale Tool

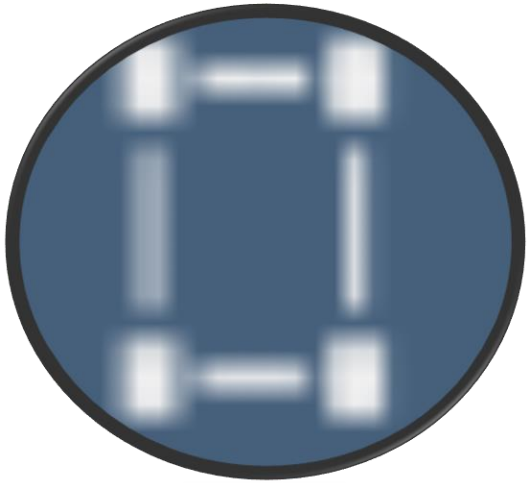
- این ابزار به شما امکان می دهد اندازه (مقیاس) اشیاء را در امتداد محورهای خاصی تغییر دهید.

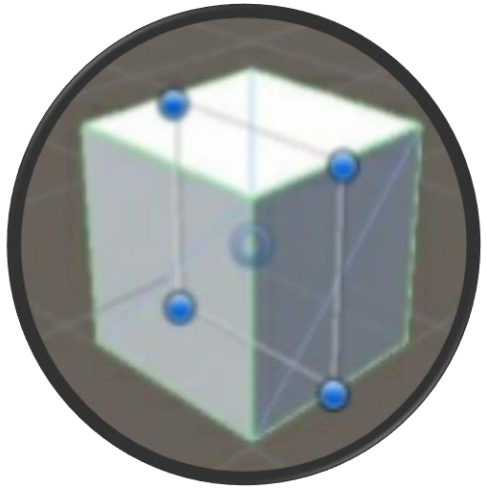




Rect Tool

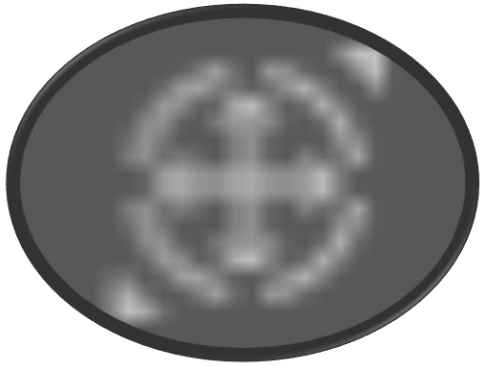
- این ابزار یک ابزار همه کاره است و مانند ترکیبی از ابزار Moveو Scalingرفتار می کند ، اما مستعد از بین رفتن دقت است.
- در تنظیم عناصر UI مفیدتر است.





Transform Tool

- تغییراتی مانند موقعیت،چرخش ومقیاس در این قسمت اتفاق میافتد.



اصطلاحات کلیدی در یونیتی

Asset

۱. **Asset** ها منابع بازی هستند.

۲. تمام چیزهایی که در بازی وجود دارند **assest** هستند، مثل(کد، تکسچر، صدا ،مدل سه بعدی و....).

۳. **Asset** ها در تب **Project** نمایش داده میشوند.

۴. هر **Asset** ویژگی هایی دارد که با انتخاب آن در تب **Inspector** نمایش داده میشوند وبرای تغییر ویژگیها نیز می توان از این تب استفاده کرد.

Asset

۵. برای **مدیریت راحت تر** Asset ها باید آنها را پوشه بندی کرد.
۶. برای **وارد کردن** یک Asset می توانیم در پنجره Project کلیک راست کرده و گزینه Import New Assets را انتخاب کنیم تا شی مورد نظر وارد یونیتی شود.
۷. برخی از Assetها باید در خود یونیتی ساخته شوند، مانند متریال،کدها،شیدرها و ....
۸. برای **ساخت** Asset ها، در پنجره Project ،راست کیک کرده و سپس از گزینه Create ، Asset مورد نظر را انتخاب میکنیم.

Package

- به مجموعه ای از **Asset ها** که در یک فایل قرار دارد **Package** می گویند.
- یکی از کاربردهای **Package**، استفاده از **Asset‌های** موجود در آن در چندین پروژه است. به عنوان مثال تمام کدها، اشیاء و سایر Asset‌های مورد نیاز برای یک ماشین را در یک **Package** قرار میدهیم، حال اگر در پروژه های بعدی اگر نیازی به استفاده از ماشین داشتیم میتوانیم از این **Package** آماده شده استفاده کرد.

- معمولاً یونیتی چندین **Package** آماده دارد که میتوان از آنها استفاده کرد.

Game Object

- **Game Object** مهمترین چیز در یونیتی است.
- هر چیزی که در بازی وجود دارد یک **Game Object** است.
- **Game Object** به خودی خود هیچ عملی انجام نمی دهد و با اضافه شدن **Component ها** ویژگی خاصی پیدا میکنند.
- در یونیتی **Game Object** هایی آماده ، ساخته شده است که میتوان آن ها را از منوی **GameObject** و انتخاب **Create Other Game Object** ایجاد کرد، قابل ذکر است که **Game Object** آماده ،خود نیز متشکل از یک **Empty game object** بعلاوه یکسری **Component** است.

Game Object

- ما بسته به نیاز بازی خود این Game Object ها را میسازیم .
- Game Objectهای یونیتی در لیست زیر آمده اند:

- Spotlight
- Area light
- Cube
- Sphere
- Capsule
- Cylinder
- Plane
- Quad
- Sprite
- Cloth
- Terrain
- Ragdoll
- Tree
- Wind zone

sprite

Sprite Mode :

با استفاده از گزینه می‌توان تعیین کرد که فایل تنها شامل:

- یک اسپرایت است (Single)
- چندین اسپرایت را در خود جای داده است
  - (Multiple)
  - (Polygon)

- قرار دادن چندین اسپرایت بازی در یک فایل تصویری امری متداول است.
- این کار برای کاهش حجم بازی و کاهش تعداد فراخوانی‌های مرتبط با رندر (drawcall) صورت می‌پذیرد.
- به تصاویری که چندین اسپرایت در آن قرار دارند texture atlasگفته می‌شود.
- تفکیک کردن spriteهای یک texture atlas از طریق ابزاری به نام Sprite Editorصورت می‌پذیرد .

Component

- **Component** ها ویژگی هایی هستند که ما به یک Game Object میدهیم تا ویژگی خاصی در بازی داشته باشد.
- برای مثال به آن Collider می دهیم تا در بازی دارای فیزیک باشد یا به آن **Audio Source** می دهیم تا در بازی از خود صدایی پخش کند و یا خیلی کارهای دیگر که بعدا با بسیاری از این Component ها آشنا میشویم.
- برخی از کامپوننت های اصلی یونیتی عبارتند از:

1. collider
2. Physics ۲d
3. Audio
4. Scripts

Prefab

- هر **Game Object** که در صحنه ای از بازی ساخته میشود، مختص همان صحنه است و نمیتوان در صحنه های دیگر بازی از آن استفاده کرد.پس ما باید آن را درون یک **Prefab** قرار دهیم تا هم آن را در صحنه های دیگر استفاده کنیم و یا اگر تغییری در **Prefab** ایجاد کنیم تمامی نمونه هایی که از این **Prefab** در بازی قرار گرفته است نیز تغییر میکند.
- مفهوم **Prefab** تقریبا شبیه به **کلاس در برنامه نویسی** است به طوری که ما یک **Prefab** اصلی ایجاد میکنیم و نمونه هایی از آن را در بازی قرار می دهیم.حال با تغییر **Prefab** اصلی تمامی نمونه ها نیز تغییر می کند.

شناخت و کار با دوربین

معرفی دوربین

- دوربین ها ،صحنه ها را رندر میکنند، بطوریکه کاربر بتواند انها را ببیند.
- هرصحنه باید حداقل یک دوربین داشته باشد.
- در هنگام ایجاد یک صحنه جدید ،یونیتی یک دوربین به نام Main Camera ایجاد میکند.
- یک صحنه میتواند چندین دوربین داشته باشد.
- دوربین ها ،اشیاء بازی هستند و میتوانند با استفاده از ابزار تغییر شکل ونیز در نمای inspector ویرایش شوند.

دسته بندی دوربین ها،براساس تصاویر دوربین

۱. **Perspective** : صحنه ها را براساس زاویه دوربین به شکلی که در صحنه موجود است،رندر میگیرد.

۱. **Orthographic** : صحنه به طور یکنواخت و بدون هیچ دورنمای سه بعدی رندر میگیرد.از این نوع دوربین معمولا برای بازی های از بالا به پایین استفاده میشود.

فیزیک در یونیتی

- یونیتی دارای موتور فیزیکی NVIDIA PHYSIX است.این موتور اجازه ایجاد رفتارهای فیزیکی بی نظیر را میدهد و همچنین خصوصیات مفید دیگری دارد.
- به وسیله کنترل فیزیک توسط کدها میتوان به یک شئ تحرک بخشید.(برای مثال حرکات جزئی از یک لباس ،یا برخوردهایی که با ماشین ایجاد میشود و...)
- فیزیک در یونیتی شامل colliderها و rigidbody است.

Collider

- Collider ها مهمترین بخش یک فیزیک هستند که مشخص کننده ناحیه برخورد فیزیکی جسم را مشخص میکنند .
- اگر شئی دارای Collider نباشد، یعنی جسم نیست و این به این معنی است که اشیای دارای collider در هنگام برخورد با آن شی از خود هیچ رفتاری بروز نمی دهند و از آن عبور می کنند.

انواع Collider

- در زیر انواع **Collider** آمده است:

۱. **Box Collider :** برای اشکال ٤ ضلعی و مکعبی شکل

۲. **Capsule Collider :** برای اشکال استوانه ای یا کپسول شکل (در برخی از بازی های اول شخص First Person) مدل بازیکن به شکل کپسول می باشد.)

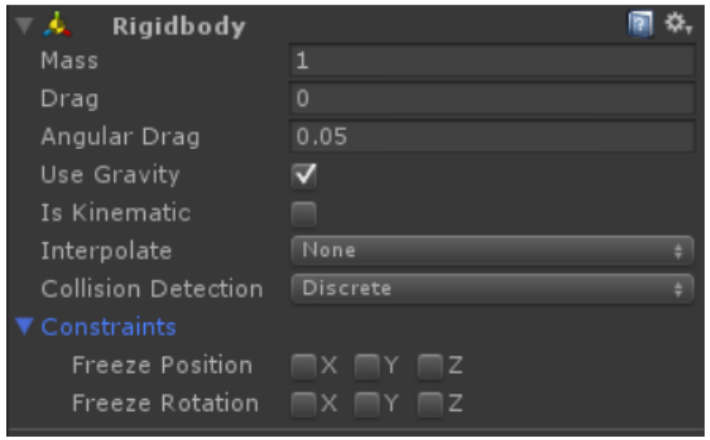
۳. **Circle collider :** برای اجسام دایره ای شکل

۴. **Sphere Collider :** برای اشکال کُره ای شکل

۵. **Wheel Collider :** این کلایدر با همه فرق می کند و برای چرخ ماشین استفاده می شود که قابلیت های اضافه ای نسبت به کلایدر های دیگر دارد.

Rigidbody

- Rigidbody شی شما قادر میسازد تا تحت کنترل فیزیک عمل کند.
- Rigidbody میتواند نیرو و گشتاور را به منظور حرکت دادن شی در یک فضای طبیعی دریافت کند .
- همه اشیاء باید شامل Rigidbody باشند تا از نیروی جاذبه تاثیر بپذیرند.



مقدمات کد نویسي در یونیتي تحت C#

زبانهای برنامه نویسی در یونیتی

- یونیتی یک پلتفرم محبوب توسعه بازی است که توسط میلیون ها بازی ساز در سراسر جهان استفاده می شود.
- این پلتفرم ابزارها و ویژگی‌های مختلفی را برای توسعه بازی‌ها برای پلتفرم‌های مختلف از جمله دستگاه‌های موبایل، دسکتاپ، کنسول و دستگاه‌های VR/AR ارائه می‌کند.
- Unityامروزه از زبان های C#، C/C++، Rust، IronPythonو Lua پشتیبانی می کند.

تفاوت فیلد و متغیر چیست؟

```
public class User : IUser
{
  //fileds
  private int id;
  private string name;

  public int id
  public string name;

public void PrintUserInfo()
        {
         //local variable
          int i = 100;
          float floatVar = 10.2f;
          char charVar = 'A';
          bool boolVar = true;
        }
}
```

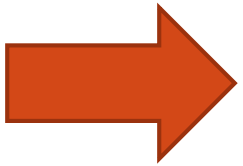
- فیلد (field) ها در واقع متغیر (variable)های عضو یک کلاس هستند که به صورت مستقیم در خود کلاس تعریف می شوند و در سطح آن در دسترس هستند.
- اگر این متغیر ها به صورت مستقیم داخل بدنه یک متد تعریف شوند به آنها local variableگفته می شود.
- طبق اصول برنامه نویسی شئ گرا، دنیای خارج از یک کلاس نباید از عملیات صورت گرفته داخل آن کلاس با خبر باشد، برای همین، فیلدها که از مشخصه های یک کلاس هستند معمولا به صورت private تعریف می شوند و هرجا که نیاز باشد دنیای خارج بتواند مقادیر آنها را مشاهده کند و یا تغییری در آنها صورت دهد از پراپرتی ها استفاده می کنیم یا آن فیلد را public تعریف می کنیم.
- حرف اول فیلد ها و متغیر ها را با حروف کوچک و به صورت camel case می نویسیم. یعنی کلمه اول با حرف کوچک واولین حرف کلمه بعدی باید بزرگ باشد.

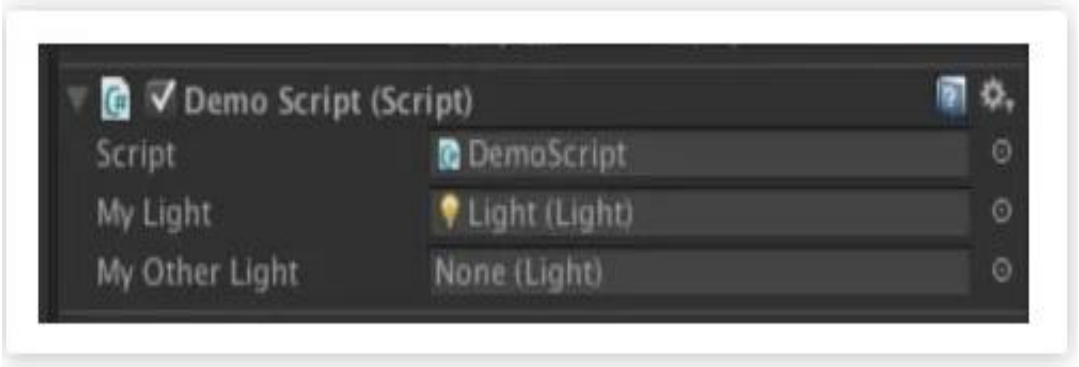
متغیر ها

- متغیرها مقادیر و ارزش های اشیا را در خود جای میدهند.
- آنها مثل یک جعبه می مانند که چیزی برای استفاده در آن ها نگه داشته می شود.
- همیشه متغیرها با حرف **کوچک** نوشته می شوند.
- نام گذاری متغیر ها **نباید با عدد شروع** شود و **نباید فاصله** داشته باشد.
- متغیرها دارای انواع int,float,string,gameobject,... باشند.
- متغیرها میتوانند عمومی (public) یا خصوصی(private) باشند.

❖ وقتی editor،کد را اجرا می کند متغیر های public به شکل زیر نمایش داده میشوند، اما private ها نمایش داده نمیشوند :

```
5 public class DemoScript: MonoBehaviour {
6
7     public Light myLight;
8     private Light myOtherLight;
9
10 }
```





تفاوت متغیرهای public وprivate

- متغیر های privateکدنویسی تمیز تر را امکان پذیر می کنند. خطایابی و نگه داری از کد ها کار راحت تری خواهد بود.
- در صورت انتخاب نوع publicشما باید در تمامی پایگاه داده برای پیدا کردن کدهای خاص وقت صرف کنید اما دسترسی به این کد برای دیگران امکان پذیر خواهد بود.
- یکی دیگر از جنبه های **متغیر**ها نوع آن ها می باشند. نوع متغیر، تعیین کننده ی شکل قرار گیری در حافظه است. نوع می تواند عدد، متن و یا ترکیبی از هر دو باشد.
- **یونیتی** باید از قبل بداند که چه نوع **متغیر**ی مورد استفاده قرار می گیرد.

توابع(متدها)

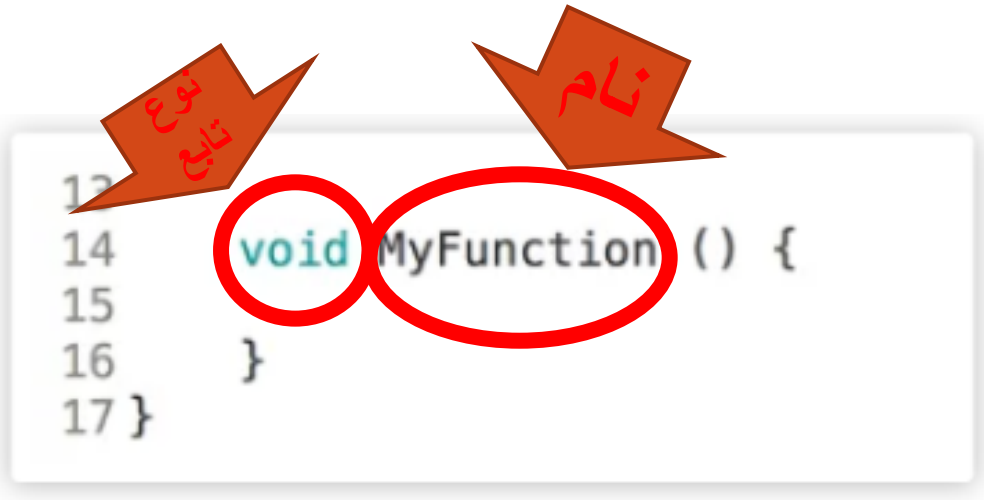
- توابع می توانند محاسبات را انجام داده و یک مقدار را به عنوان خروجی ارائه کنند.
- **توابع** ,مجموعه ای از کد ها هستند که **متغیر** ها را دستکاری می کنند.
- **تابع** ها با حرف بزرگ شروع می شوند.
- کدها در توابع دسته بندی می شوند و می توان به راحتی آن ها را در چند جای برنامه مورد استفاده قرار داد.
- تابع های مختلفی به شکل خودکار در یونیتی اجرا می شوند،مانند توابع زیر:

```
/*
    Awake()
    Start()
    Update ()
    FixedUpdate()
    LateUpdate

*/
```

نوشتن توابع

- وقتی یک **تابع** را می نویسید، به یاد داشته باشید که ابتدا نوع تابع در ابتدا مشخص شده باشد و بعد نام آن و پارامتر های دیگر.
- نام تابع با حرف بزرگ شروع می شود. مثال:



فراخوانی توابع

- شما می توانید از یک **تابع** بخواهید تا اطلاعاتی را برای شما پردازش کند و یک جواب را به عنوان برگشتی بدهد.
- اما اگر شما از نوع " void " استفاده کنید، هیچ جوابی در برگشت ارائه نخواهد شد.

```
19
20 void MyFunction () {
21         myLight.enabled = !myLight.enabled;
22 }
23 }
```

کلاس ها

- **کلاس ها** مجموعه ای از **متغیر** ها و توابع می باشند.
- روشی برای ساختاربندی کد ها، به شکلی است که توابع و **متغیر** ها با هم یک تمپلیت (یا قالب) را ایجاد کنند که خصیصه های یک شی را معین می کند.
- منطق این کار رسیدن به یک هدف مشخص است.
- توجه داشته باشید که نام کلاس باید با نام فایل **C #** هم خوانی داشته باشد.
- هم چنین برای این که به یک gameObject متصل شود باید از یک **کلاس** دیگر به نام MonoBehaviour گرفته شود که به شکل خودکار در ابتدای کدنویسی برای شما قرار داده می شود.
- کلاس ها ممکن است **public** یا **private** باشند.

نمونه ای از یک کلاس

```
 6 public class DemoScript: MonoBehaviour {
 7
 8     public Light myLight;
 9
10     void Awake () {
11         int myVar = AddTwo(9,2);
12         Debug.Log(myVar);
13     }
14
15     void Update () {
16         if (Input.GetKeyDown ("space")) {
17             MyFunction ();
18
19         }
20
21     }
22
23     void MyFunction () {
24
25         myLight.enabled = !myLight.enabled;
26     }
27
28     int AddTwo (int var1, int var2) {
29         int returnValue = var1 + var2;
30         return returnValue;
31
32     }
33
34 }
```